

RISC-V Based Configurable Data Compression Accelerator SoC

1 Introduction

1.1 Background

Modern embedded systems increasingly require the processing of large amounts of digital data generated by sensors, communication interfaces, storage devices, and other peripherals. As the amount of data increases, storing and transferring the data can require significant memory capacity and communication bandwidth. One approach to reducing the amount of data that needs to be stored or transferred is **lossless data compression**.

Lossless compression techniques reduce the representation size of data while preserving all of the original information. Unlike lossy compression, no information is intentionally discarded during the compression process. The original data can therefore be reconstructed exactly from the compressed representation.

In many conventional embedded systems, compression is performed entirely by software running on a general-purpose processor. Although this approach is flexible, repeatedly executing compression algorithms on the processor can consume processor cycles and increase the time required for data processing. A dedicated hardware accelerator can instead perform computationally repetitive operations directly in hardware, allowing the processor to focus on system control and other tasks.

This project proposes a **RISC-V based System-on-Chip (SoC)** that integrates a dedicated hardware accelerator for **Run-Length Encoding (RLE)** compression and decompression.

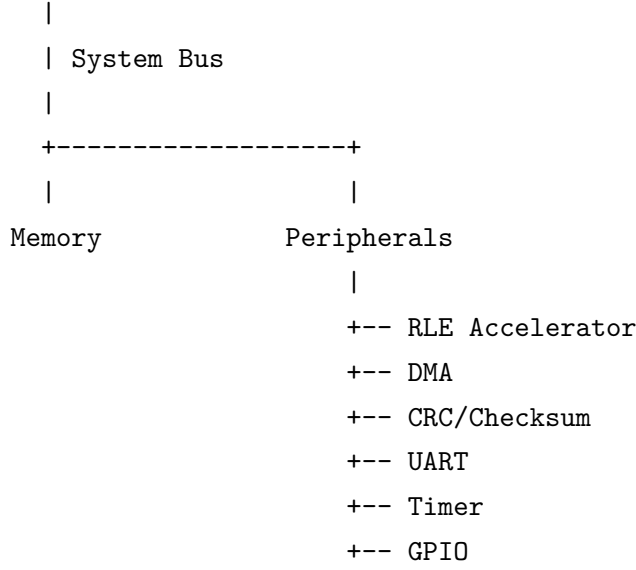
1.2 System-on-Chip Concept

A **System-on-Chip (SoC)** integrates the major components required for an embedded computing system into a single hardware design. Instead of treating the processor, memory, peripherals, and specialized hardware as completely separate systems, they are interconnected within one SoC.

The proposed SoC contains a RISC-V processor as its main processing element. The processor executes software and controls the other hardware components through a system interconnect.

The basic concept is:

RISC-V Processor



The processor is responsible mainly for **control and configuration**, while the dedicated RLE accelerator performs the compression and decompression operation.

1.3 RISC-V Processor

The processor selected for the proposed SoC is the **VeeR EL2 RISC-V processor core**.

VeeR EL2 is an open-source RISC-V processor core designed for embedded applications. According to the VeeR EL2 Programmer's Reference Manual, the core supports the RV32IMC instruction set configuration and provides multiple system-bus interfaces, including interfaces associated with instruction fetching, load/store operations, debug access, and DMA. The system-bus interface can be configured to use AXI4 or AHB-Lite.

In this project, the VeeR EL2 processor acts as the **central controller of the SoC**.

The processor executes software that performs tasks such as:

- system initialization,
- preparation of input data,
- configuration of the RLE accelerator,
- configuration of data transfers,
- starting compression or decompression,
- monitoring accelerator status,
- initiating integrity checks,

- checking the results, and
- reporting results through a communication interface such as UART.

The processor does not need to perform every individual RLE operation itself. Instead, it communicates with the accelerator through memory-mapped control registers.

1.4 Run-Length Encoding

Run-Length Encoding (RLE) is a simple lossless data compression technique that represents consecutive repeated values using a value and its repetition count.

For example, consider the input sequence:

A A A A B B C C C

Instead of storing every value individually, RLE can represent it as:

A 4 B 2 C 3

This means:

- A occurs 4 consecutive times,
- B occurs 2 consecutive times, and
- C occurs 3 consecutive times.

The compressed representation can therefore require less storage when the input contains long sequences of repeated values.

The reverse operation is **RLE decompression**. For example:

A 4 B 2 C 3

is expanded back to:

A A A A B B C C C

The original sequence is recovered exactly.

Therefore, the proposed accelerator supports two operating modes:

```

Compression:
Original Data -> RLE -> Compressed Data

Decompression:
Compressed Data -> RLE -> Original Data

```

1.5 Motivation for Hardware Acceleration

RLE itself is a relatively simple algorithm, but it contains repetitive operations such as:

- reading input values,
- comparing consecutive values,
- maintaining a run counter,
- generating encoded output,
- repeating values during decompression, and
- managing data movement.

When these operations are performed entirely by software, the processor must execute instructions repeatedly for each input element.

A hardware accelerator can implement these operations using dedicated registers, comparison logic, counters, state machines, and data-path logic.

The proposed approach therefore divides the work between software and hardware:

```

Software / CPU
  |
  | Configuration and control
  v
RLE Hardware Accelerator
  |
  | Actual compression/decompression
  v
Compressed / Recovered Data

```

This demonstrates an important SoC design concept: **using a general-purpose processor for system control while offloading a specific computational task to dedicated hardware.**

1.6 Proposed Application

The proposed system is intended as a hardware platform for demonstrating **data reduction and data processing in embedded systems**.

A practical application scenario is the handling of data that contains repeated or slowly changing values. Such data can occur in areas such as:

- sensor data storage,
- embedded data logging,
- memory-efficient data storage,
- low-bandwidth data transmission,
- communication buffers, and
- simple telemetry systems.

For example, consider an embedded sensor system that repeatedly records the same value over several consecutive samples:

25 25 25 25 25 26 26 26 27 27

Instead of storing every sample independently, the sequence can be represented as:

25 x 5
26 x 3
27 x 2

This representation can reduce the amount of data that needs to be stored or transferred when repeated values are common.

The proposed SoC provides a hardware-based mechanism for performing this transformation.

1.7 Why RLE is Suitable for the Proposed SoC

RLE is particularly suitable for a small hardware accelerator because its fundamental operation is straightforward.

For compression, the hardware mainly needs to:

1. read a value,
2. compare it with the previous value,
3. increment a counter when the values are equal,

4. output the previous value and count when the value changes, and
5. handle the final run.

For decompression, the hardware needs to:

1. read a value and its count,
2. output the value repeatedly,
3. decrement the count,
4. load the next encoded pair when the count reaches zero.

These operations can be implemented using relatively simple RTL structures such as:

- registers,
- counters,
- comparators,
- multiplexers,
- finite-state machines, and
- control logic.

This makes RLE appropriate for demonstrating a **custom hardware IP integrated into a processor-based SoC** without requiring a highly complex compression algorithm.

1.8 Proposed SoC

The proposed SoC combines the following major components:

1.8.1 Processor

VeeR EL2 RISC-V processor

Acts as the main controller and executes the application software.

1.8.2 RLE Accelerator

The primary custom hardware IP.

Performs:

- RLE compression
- RLE decompression

1.8.3 DMA Controller

DMA stands for **D**irect **M**emory **A**ccess.

It is intended to assist in transferring blocks of data between memory and the processing hardware without requiring the processor to manually handle every data transfer.

The exact DMA implementation and interface will be selected based on the available SoC/IP environment.

1.8.4 CRC/Checksum Engine

Used to perform a data-integrity check.

For example:

Original Data	
	v
CRC	
	v
CRC_1	
Recovered Data	
	v
CRC	
	v
CRC_2	
CRC_1 == CRC_2	
	v
PASS	

This provides an additional mechanism for confirming that the data recovered after compression and decompression matches the expected data.

1.8.5 Memory

Memory stores:

- processor instructions,
- input data,
- compressed data, and
- decompressed/recovered data.

1.8.6 UART

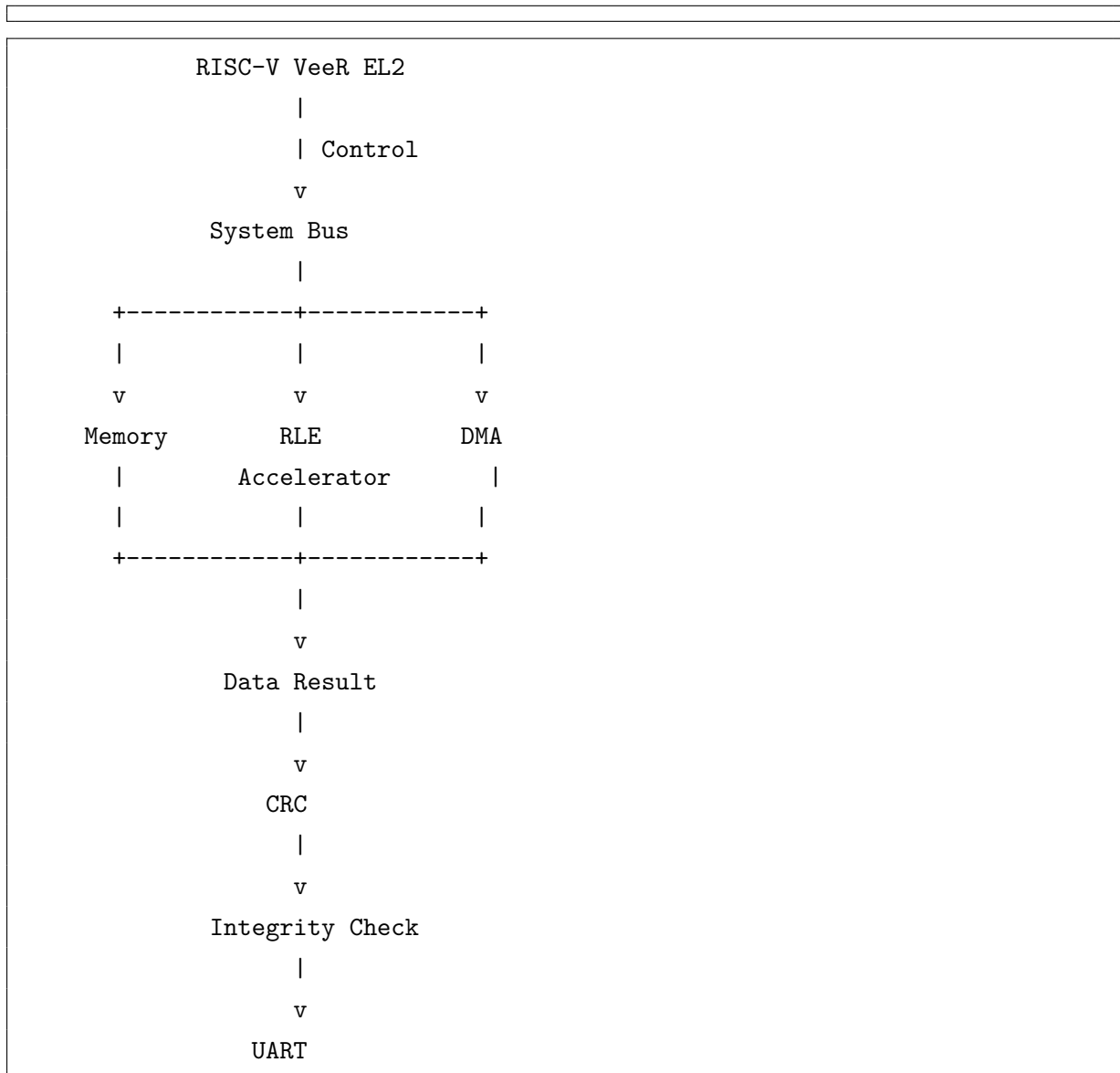
UART can be used to communicate test results or status information from the SoC to the outside system.

1.8.7 Timer and GPIO

Timer and GPIO provide basic SoC peripheral functionality and can also be used during system testing and demonstration.

1.9 Overall Operating Principle

The complete system operates approximately as follows:



A typical compression and verification sequence is:

-
1. Processor initializes the SoC.
 2. Input data is stored in memory.
 3. The processor configures the RLE accelerator.
 4. Compression is started.
 5. The RLE hardware processes the input data.
 6. Compressed data is stored in memory.
 7. The compressed data can then be decompressed.
 8. The recovered data is stored in memory.
 9. CRC/checksum operations are performed on the required data.
 10. The processor compares the results.
 11. The final status is reported.

The processor therefore provides the **control path**, while the RLE accelerator and DMA participate in the **data path**.

2 Block Diagram

The proposed RISC-V based configurable data compression accelerator SoC is organized around the VeeR EL2 processor, an AXI4 system interconnect, memory, the custom RLE compression/decompression accelerator, DMA, CRC/checksum logic, and basic peripherals. The block diagram below presents the overall system organization and the principal control, data, interrupt, clock, and reset relationships.

2.1 Overall SoC Block Diagram

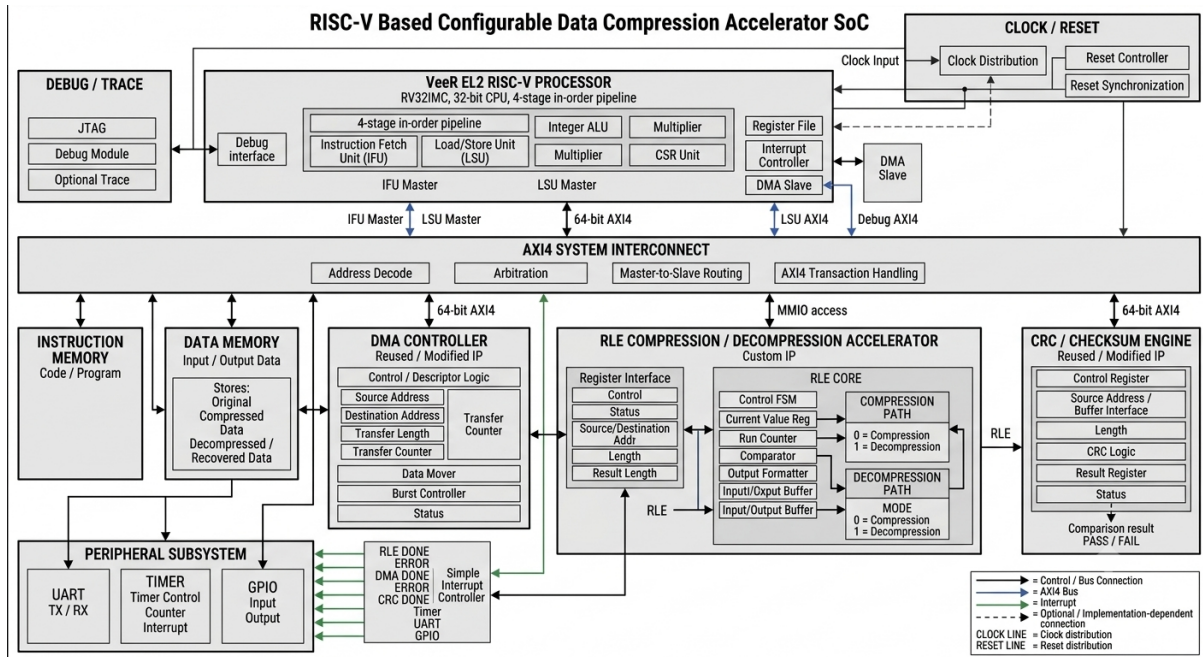
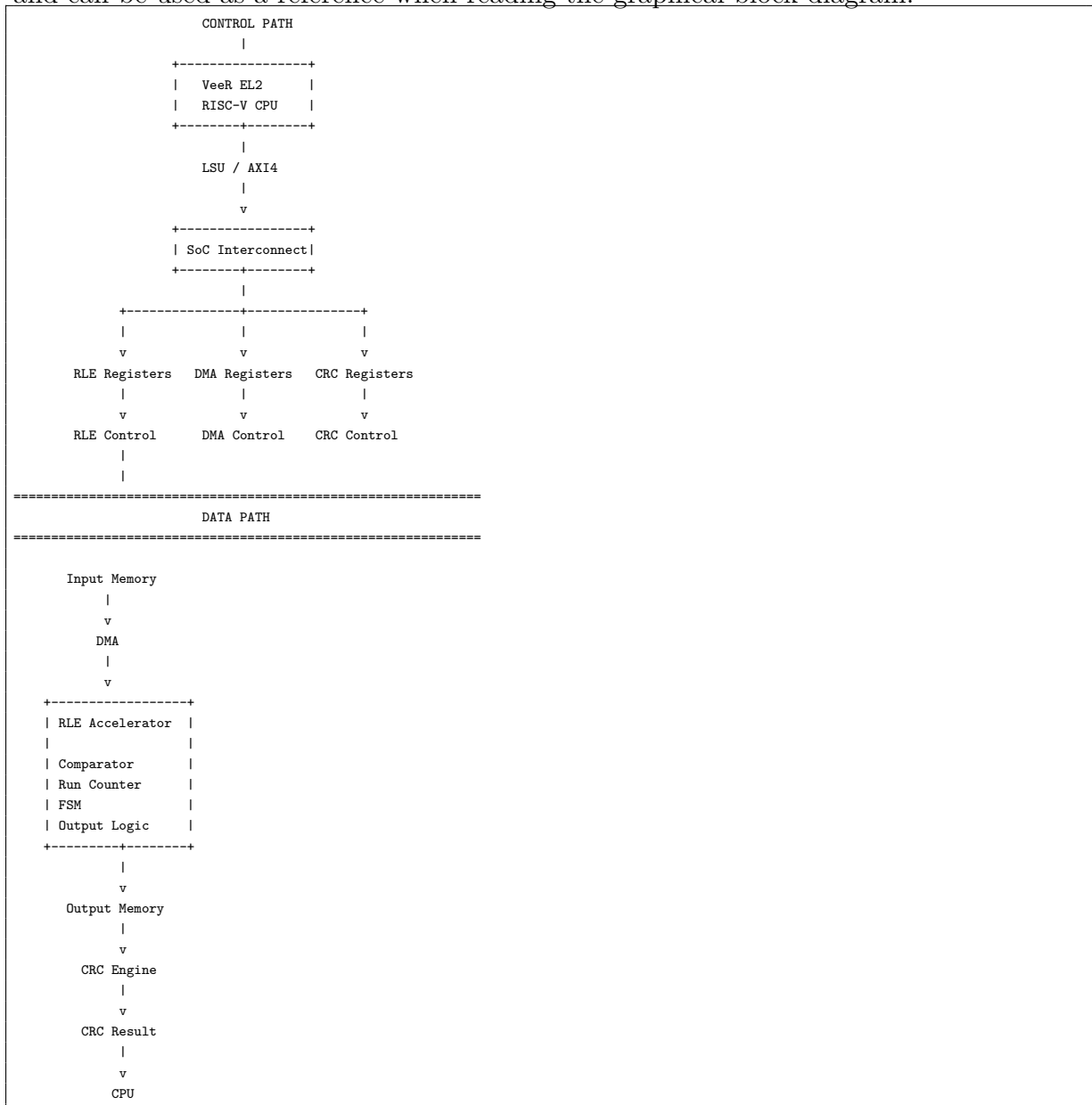


Figure: Overall block diagram of the proposed RISC-V based configurable data compression accelerator SoC

2.2 Textual Architecture Reference

The following textual representation captures the same architecture at a simplified level and can be used as a reference when reading the graphical block diagram.



2.3 Proposed Address Map

The following address map is optional and implementation-dependent. It is included to show a possible memory-mapped organization of the proposed SoC.

0x0000_0000	Instruction Memory
0x0004_0000	Data Memory
0x1000_0000	UART
0x1000_1000	Timer

0x1000_2000	GPIO
0x1001_0000	RLE
0x1001_1000	DMA
0x1001_2000	CRC

2.4 Diagram Legend

SOLID BLACK ARROW	= Control / Bus Connection
BLUE ARROW	= 64-bit AXI4 Bus
GREEN ARROW	= Interrupt Path
DOTTED CONNECTION	= Optional / Implementation-dependent connection
CLOCK LINE	= Clock distribution
RESET LINE	= Reset distribution

3 Signal List

3.1 Overview

The proposed SoC contains signals associated with:

1. clock and reset,
2. processor control and interrupts,
3. VeeR EL2 system-bus interfaces,
4. RLE accelerator control and status,
5. RLE data interface,
6. DMA control and data movement,
7. CRC/checksum operation,
8. UART,
9. GPIO, and
10. Timer.

The signals can be broadly divided into two categories:

3.1.1 Control Signals

These configure and control the operation of the hardware.

```
start
mode
source address
destination address
length
enable
reset
```

3.1.2 Data and Bus Signals

These carry addresses, data, and transaction information.

```
address
write data
read data
valid
ready
response
byte strobes
```

3.2 Clock and Reset Signals

The VeeR EL2 core complex has a main clock input and individual clock-enable inputs for its system-bus interfaces. These are defined in the PRM.

Signal	Direction	Width	Description
clk	Input	1	Main SoC/core clock
ifu_bus_clk_en	Input	1	Enables the IFU bus clock
lsu_bus_clk_en	Input	1	Enables the LSU bus clock
dbg_bus_clk_en	Input	1	Enables the debug bus clock
dma_bus_clk_en	Input	1	Enables the DMA slave bus clock
rst_l	Input	1	VeeR core-complex reset
rst_vec[31:1]	Input	31	Core reset vector
dbg_rst_l	Input	1	Debug-module reset

Table 1: Clock and reset signals

The VeeR documentation explicitly defines these clock and reset signals.

For the rest of the SoC, the same clock domain can initially be used for the RLE, CRC, DMA, and basic peripherals unless the final implementation requires separate clock domains.

3.3 VeeR Interrupt Signals

The VeeR core provides interrupt inputs including:

Signal	Direction	Width	Purpose
nmi_int	Input	1	Non-maskable interrupt
nmi_vec[31:1]	Input	31	NMI vector
soft_int	Input	1	Software interrupt
timer_int	Input	1	Timer interrupt
extintsrc_req[...]	Input	Configurable	External interrupt requests

Table 2: VeeR interrupt signals

These are defined by the VeeR EL2 core interface.

For our project, the RLE accelerator does not need to depend on an interrupt initially. The simplest implementation is for the processor to poll the RLE `STATUS` register.



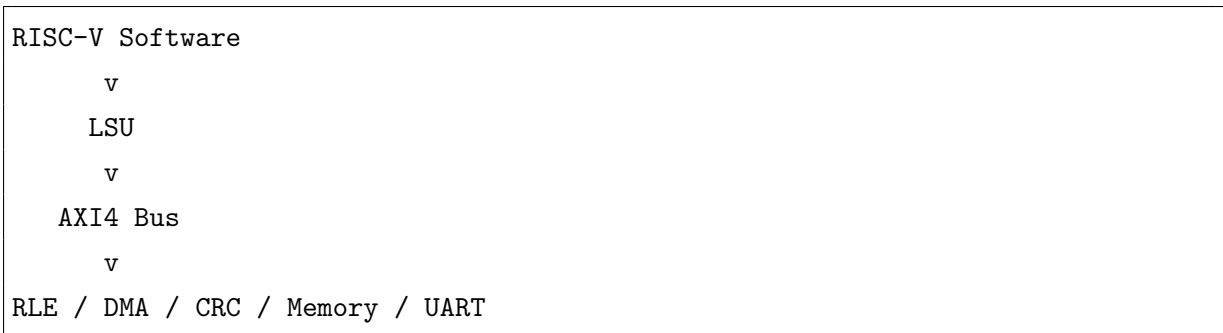
An interrupt can be added later if required.

3.4 VeeR EL2 LSU AXI4 Interface

The **Load/Store Unit (LSU)** is the most important processor interface for our accelerator.

The CPU uses the LSU to perform memory and peripheral accesses.

Conceptually:



The VeeR PRM defines the LSU AXI4 interface as a 64-bit data interface with separate AXI channels.

3.4.1 LSU Write Address Channel

Signal	Direction	Width	Purpose
lsu_axi_awvalid	Output	1	CPU has a valid write address
lsu_axi_awready	Input	1	Slave/interconnect accepts address
lsu_axi_awid	Output	LSU_BUS_TAG	Transaction ID
lsu_axi_awaddr	Output	32	Write address
lsu_axi_awlen	Output	8	Burst length
lsu_axi_awsz	Output	3	Transfer size
lsu_axi_awburst	Output	2	Burst type
lsu_axi_awlock	Output	1	Lock information
lsu_axi_awcache	Output	4	Memory/cache attributes
lsu_axi_awprot	Output	3	Protection information
lsu_axi_awqos	Output	4	Quality-of-service information
lsu_axi_awregion	Output	4	Region identifier

Table 3: LSU AXI4 write address signals

The PRM identifies the burst length, burst type, protection, and other attributes, including hardwired values for some LSU signals.

3.4.2 LSU Write Data Channel

Signal	Direction	Width	Purpose
lsu_axi_wvalid	Output	1	Write data is valid
lsu_axi_wready	Input	1	Slave/interconnect accepts write data
lsu_axi_wdata	Output	64	Data being written
lsu_axi_wstrb	Output	8	Byte-enable/write strobes
lsu_axi_wlast	Output	1	Indicates last write transfer

Table 4: LSU AXI4 write data signals

The 64-bit `wdata` and 8-bit `wstrb` are explicitly defined in the PRM.

3.4.3 LSU Write Response Channel

Signal	Direction	Width	Purpose
lsu_axi_bvalid	Input	1	Write response is valid
lsu_axi_bready	Output	1	CPU accepts response
lsu_axi_bid	Input	LSU_BUS_TAG	Response transaction ID
lsu_axi_bresp	Input	2	Write response/status

Table 5: LSU AXI4 write response signals

These signals allow the processor to know whether its write transaction completed successfully.

3.4.4 LSU Read Address Channel

Signal	Direction	Width	Purpose
lsu_axi_arvalid	Output	1	CPU has a valid read request
lsu_axi_arready	Input	1	Slave/interconnect accepts request
lsu_axi_arid	Output	LSU_BUS_TAG	Read transaction ID
lsu_axi_araddr	Output	32	Read address
lsu_axi_arlen	Output	8	Burst length
lsu_axi_arsize	Output	3	Transfer size
lsu_axi_arburst	Output	2	Burst type
lsu_axi_arlock	Output	1	Lock information
lsu_axi_arcache	Output	4	Memory/cache attributes
lsu_axi_arprot	Output	3	Protection information
lsu_axi_arqos	Output	4	QoS information
lsu_axi_arregion	Output	4	Region identifier

Table 6: LSU AXI4 read address signals

These are the VeeR-defined LSU AXI4 read-address signals.

3.4.5 LSU Read Data Channel

Signal	Direction	Width	Purpose
lsu_axi_rvalid	Input	1	Read data is valid
lsu_axi_rready	Output	1	CPU accepts read data
lsu_axi_rid	Input	LSU_BUS_TAG	Read transaction ID
lsu_axi_rdata	Input	64	Read data
lsu_axi_rresp	Input	2	Read response
lsu_axi_rlast	Input	1	Last read transfer

Table 7: LSU AXI4 read data signals

The VeeR PRM defines these signals and their 64-bit read-data width.

3.5 IFU AXI4 Interface

The **Instruction Fetch Unit (IFU)** obtains instructions for the processor.

Conceptually:

Instruction Memory

~



The IFU interface uses the same AXI4 channel structure, with its own signal prefix:

ifu_axi_*

The PRM defines the IFU address, data, response, and read channels. The IFU is read-only from a functional point of view; its write-channel signals are present but disabled/tied off.

Important IFU signals include:

ifu_axi_arvalid
ifu_axi_arready
ifu_axi_arid
ifu_axi_araddr
ifu_axi_arlen
ifu_axi_arsize
ifu_axi_arburst
ifu_axi_arlock
ifu_axi_arcache
ifu_axi_arprot
ifu_axi_arqos
ifu_axi_arregion

ifu_axi_rvalid
ifu_axi_rready
ifu_axi_rid
ifu_axi_rdata
ifu_axi_rresp
ifu_axi_rlast

The IFU read data is 64 bits wide.

For our project, these signals are primarily connected to the instruction-memory path through the system interconnect.

3.6 Debug AXI4 Interface

VeeR also provides a separate debug system-bus master interface.

Its signals use the `sb_axi_*` prefix.

It follows the AXI4 structure:

Write Address
Write Data
Write Response
Read Address
Read Data

The PRM defines the debug AXI4 signals separately from the LSU interface.

This interface is not part of the normal RLE processing path.

It is mainly relevant for:

- debugging,
- processor inspection,
- development,
- breakpoints, and
- debug access.

Therefore, the architecture document should show it, but we do not need to describe every debug signal again in the RLE section.

3.7 VeeR DMA Slave AXI4 Interface

The VeeR EL2 core complex also provides a **DMA slave interface**.

This is important because it allows external DMA access to the VeeR core complex according to the selected memory configuration.

The interface uses signals with the following prefix:

<code>dma_axi_*</code>

The VeeR PRM defines the following groups.

3.7.1 DMA Write Address

<code>dma_axi_awvalid</code>
<code>dma_axi_awready</code>

<code>dma_axi_awid</code> <code>dma_axi_awaddr</code> <code>dma_axi_awlen</code> <code>dma_axi_awsz</code> <code>dma_axi_awburst</code> <code>dma_axi_awprot</code>
--

3.7.2 DMA Write Data

<code>dma_axi_wvalid</code> <code>dma_axi_wready</code> <code>dma_axi_wdata</code> <code>dma_axi_wstrb</code> <code>dma_axi_wlast</code>
--

3.7.3 DMA Write Response

<code>dma_axi_bvalid</code> <code>dma_axi_bready</code> <code>dma_axi_bid</code> <code>dma_axi_bresp</code>
--

3.7.4 DMA Read Address

<code>dma_axi_arvalid</code> <code>dma_axi_arready</code> <code>dma_axi_arid</code> <code>dma_axi_araddr</code> <code>dma_axi_arlen</code> <code>dma_axi_arsz</code> <code>dma_axi_arburst</code> <code>dma_axi_arprot</code>
--

3.7.5 DMA Read Data

<code>dma_axi_rvalid</code> <code>dma_axi_rready</code> <code>dma_axi_rid</code>
--

dma_axi_rdata
dma_axi_rresp
dma_axi_rlast

These signal names, directions, and widths are taken from the VeeR PRM.

Important: This interface should not automatically be interpreted as “DMA directly connected to RLE.” The final DMA architecture depends on the DMA IP we select and on the final VeeR memory configuration.

3.8 RLE Accelerator Interface

Unlike the VeeR signals above, the following are **our proposed RLE IP signals**.

The RLE accelerator will be accessed through memory-mapped registers.

The external interface can therefore be divided into:

1. Clock/reset,
2. Bus/register interface,
3. Control signals,
4. Status signals, and
5. Data-path signals.

3.8.1 RLE Clock and Reset

Signal	Direction	Width	Purpose
clk	Input	1	RLE operating clock
rst_n	Input	1	Active-low reset

Table 8: RLE clock and reset signals

3.8.2 RLE Control Registers

The processor writes the following configuration information:

Signal/Register	Width	Purpose
CTRL.START	1	Starts the RLE operation
CTRL.MODE	1	Selects compression/decompression
SRC_ADDR	32	Starting address of input data
DST_ADDR	32	Starting address of output data
LENGTH	32	Number of input bytes/elements

Table 9: RLE control registers

Conceptually:

```

CPU
|
| AXI4 write
v
RLE Register Bank
|
|-- START
|-- MODE
|-- SRC_ADDR
|-- DST_ADDR
'-- LENGTH

```

These register names are project-defined, not VeeR-defined.

3.9 RLE Status Signals

The RLE accelerator provides status information to the processor.

Register/Signal	Width	Purpose
STATUS.BUSY	1	Accelerator is processing
STATUS.DONE	1	Operation completed
STATUS.ERROR	1	Error occurred
RESULT_LEN	32	Number of output bytes/elements

Table 10: RLE status signals

Typical operation:

```

CPU writes START
    v
BUSY = 1

```

v
RLE processing
v
BUSY = 0
DONE = 1
v
CPU reads RESULT_LEN

3.10 RLE Data-Path Signals

The exact RLE memory interface will depend on how the accelerator is integrated.

For the internal RLE processing logic, the important logical signals are:

Signal	Width	Purpose
data_in	8	Input data element
data_in_valid	1	Input data is valid
data_in_ready	1	RLE can accept input
data_out	8/16*	Output encoded/decoded data
data_out_valid	1	Output data is valid
data_out_ready	1	Downstream block accepts output
last	1	Indicates end of input/output stream

Table 11: RLE data-path signals

**The final encoded output width depends on the selected RLE data format.*

For the current project, the logical data element is intended to be **8 bits**. The exact RLE element/output formatting remains an implementation item to finalize.

3.11 Internal RLE Signals

Inside the custom RLE IP, the important internal signals are not SoC ports. They are RTL signals.

3.11.1 Current Value

current_value[7:0]

Stores the value currently being counted.

3.11.2 Previous Value

```
previous_value[7:0]
```

Used for comparison with the incoming value.

3.11.3 Run Counter

```
run_count[COUNT_WIDTH-1:0]
```

Stores the number of consecutive occurrences.

3.11.4 Comparator

Logical result:

```
same_value
```

where:

```
same_value = (current_value == previous_value)
```

3.11.5 FSM State

For example:

```
IDLE  
LOAD  
PROCESS  
WRITE  
DONE
```

These are internal implementation signals, not external SoC ports.

3.12 DMA Interface

The DMA controller is intended to handle block data movement.

At the control interface, the processor needs to configure:

```
DMA_SRC_ADDR  
DMA_DST_ADDR  
DMA_LENGTH
```

DMA_CONTROL
DMA_STATUS

Conceptual signals:

Signal	Width	Purpose
dma_start	1	Start transfer
dma_src_addr	32	Source address
dma_dst_addr	32	Destination address
dma_length	32	Transfer size
dma_busy	1	Transfer in progress
dma_done	1	Transfer completed
dma_error	1	Transfer error

Table 12: Logical DMA control signals

These are logical project-level signals.

The actual DMA IP may use a completely different interface, so once an actual DMA IP is selected, its real signal names and widths should replace these proposed names.

The architecture document already identifies this as an open implementation item.

3.13 CRC/Checksum Interface

The CRC engine is also treated as a supporting IP.

The logical control interface is:

Signal/Register	Width	Purpose
CRC_START	1	Start CRC operation
CRC_SRC_ADDR	32	Address of data
CRC_LENGTH	32	Number of bytes
CRC_BUSY	1	CRC calculation active
CRC_DONE	1	Calculation completed
CRC_RESULT	32	Calculated CRC value

Table 13: CRC/checksum interface

Conceptually:

CPU



The architecture specifies CRC/checksum as an integrity-checking block, with the CPU comparing the CRC of the original and recovered buffers.

The exact CRC width, polynomial, and IP interface are not yet fixed and should not be presented as final until the selected implementation is known.

3.14 UART Signals

UART is a simple communication peripheral used to report results.

The minimum external UART signals are:

Signal	Direction	Width	Purpose
uart_tx	Output	1	Serial transmit
uart_rx	Input	1	Serial receive

Table 14: UART signals

For our application, the important path is:



Example output:

```

RLE TEST
Compression: PASS

```

Decompression: PASS
CRC CHECK: PASS

3.15 GPIO Signals

GPIO means **General Purpose Input/Output**.

A simple 32-bit GPIO interface can be represented as:

Signal	Direction	Width	Purpose
gpio_in	Input	32	External input
gpio_out	Output	32	External output
gpio_oe	Output	32	Output-enable control

Table 15: GPIO signals

GPIO is not directly involved in RLE processing.

It is a standard SoC peripheral and can be used for simple status indication or external testing.

3.16 Timer Signals

The timer is another memory-mapped peripheral.

At the external level, a timer may have:

Signal	Direction	Width	Purpose
timer_clk	Input	1	Timer clock
timer_reset	Input	1	Timer reset
timer_int	Output	1	Timer interrupt

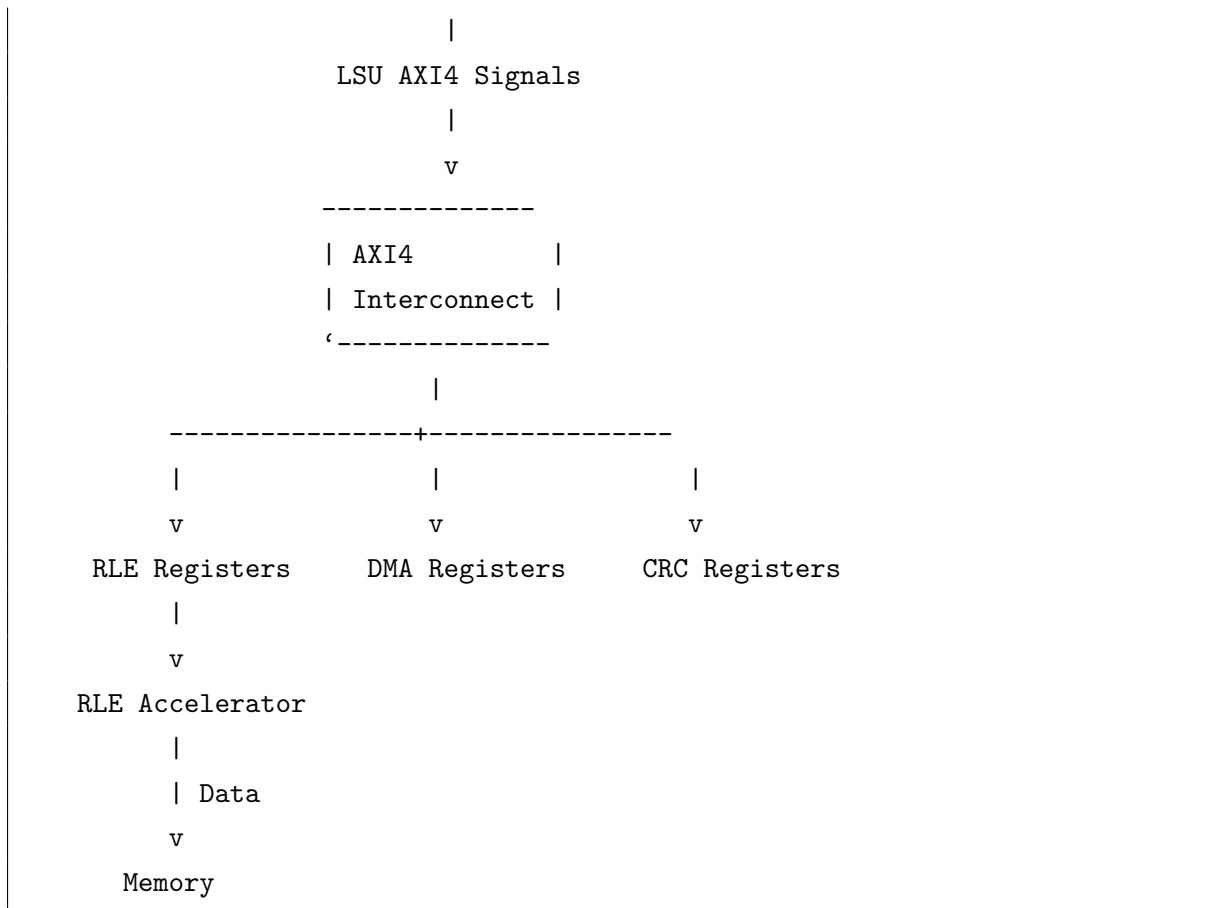
Table 16: Timer signals

The VeeR processor has a dedicated `timer_int` input for the standard timer interrupt.

3.17 System-Level Signal Relationship

The important signal relationship is:

VeeR EL2



The CPU therefore does not need separate special-purpose wires such as:

```

CPU -> RLE_START
CPU -> RLE_MODE
CPU -> RLE_DATA

```

Instead, the CPU performs normal memory-mapped writes and reads through the LSU and system bus.

For example:

```

CPU
|
| store instruction
v
LSU
|
| AXI4 write
v
Bus
|
| address = RLE_CONTROL

```

v
RLE Register Bank
'-- START = 1

This is a very important part of understanding the architecture.

3.18 Signal Categories Summary

Category	Main Signals	Source
Clock	clk, bus clock enables	VeeR / SoC
Reset	rst_l, dbg_rst_l, reset vector	VeeR
Interrupt	nmi_int, soft_int, timer_int, external interrupts	VeeR
Instruction bus	ifu_axi_*	VeeR
Data/control bus	lsu_axi_*	VeeR
Debug bus	sb_axi_*	VeeR
DMA slave bus	dma_axi_*	VeeR
RLE control	START, MODE, addresses, length	Project-defined
RLE status	BUSY, DONE, ERROR, result length	Project-defined
RLE data	input/output valid, ready, data	Project-defined
DMA control	source, destination, length, start/status	Depends on selected DMA IP
CRC control	address, length, start/status/result	Depends on selected CRC IP
UART	uart_tx, uart_rx	SoC peripheral
GPIO	input/output/enable	SoC peripheral
Timer	timer control/status/interrupt	SoC peripheral

Table 17: Signal categories summary

3.19 Important Accuracy Note

There are **three levels of signals** in this project, and we should keep them separate in the final document.

3.19.1 Level 1 — VeeR Signals

These are taken directly from the VeeR EL2 PRM.

Examples:

lsu_axi_awvalid
lsu_axi_awaddr[31:0]
lsu_axi_wdata[63:0]
lsu_axi_arvalid
lsu_axi_rdata[63:0]

The PRM explicitly defines these.

3.19.2 Level 2 — SoC/Interconnect Signals

These depend on how we connect the VeeR core to the selected memories and peripherals. They should be finalized when the actual SoC wrapper is implemented.

3.19.3 Level 3 — Custom/Project IP Signals

These are designed by us.

Examples:

RLE_START
RLE_MODE
RLE_SRC_ADDR
RLE_DST_ADDR
RLE_LENGTH
RLE_BUSY
RLE_DONE
RLE_ERROR
RLE_RESULT_LEN

These are not VeeR signals.

That distinction is important because it prevents the architecture document from looking technically impressive but containing invented VeeR ports.

For the final architecture document, this signal list should be treated as the detailed signal reference. The LSU AXI4 interface, clock/reset/interrupts, and the RLE/DMA/CRC logical interfaces are the signals that matter most to understanding the actual SoC.

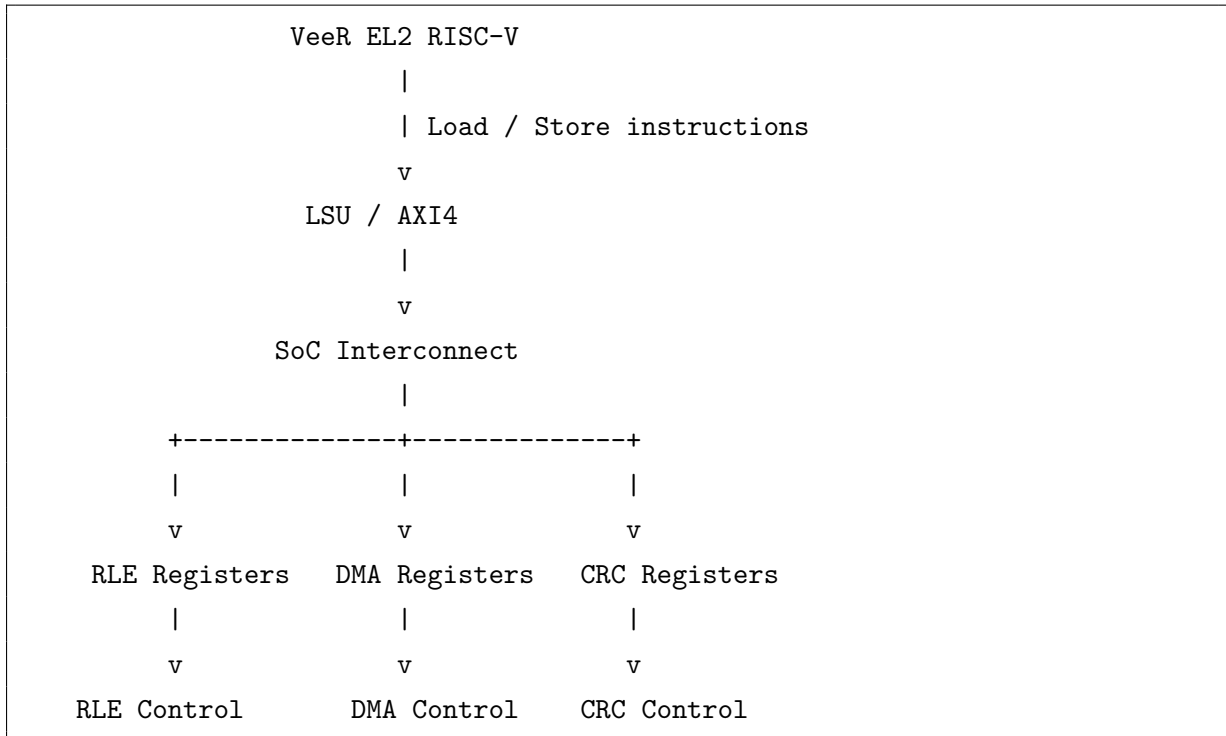
4 Control Path

The proposed SoC separates system operation into a control path and a data path. The control path tells each hardware block what operation to perform, while the data path carries the information being processed. This separation allows the VeeR EL2 processor to control the RLE accelerator without performing every compression operation itself.

4.1 Control Path Overview

The control path starts from the **VeeR EL2 RISC-V processor**. The processor executes software instructions that configure and control the RLE accelerator and supporting peripherals.

4.1.1 Control Flow



The processor can write configuration information such as:

```
RLE_SRC_ADDR
RLE_DST_ADDR
RLE_LENGTH
RLE_MODE
RLE_START
```

where:

- RLE_SRC_ADDR — address of the input data.
- RLE_DST_ADDR — address where the result is stored.
- RLE_LENGTH — amount of input data.
- RLE_MODE — compression or decompression.
- RLE_START — starts the operation.

The RLE accelerator then performs the operation independently.

After starting the operation, the processor can read status information such as:

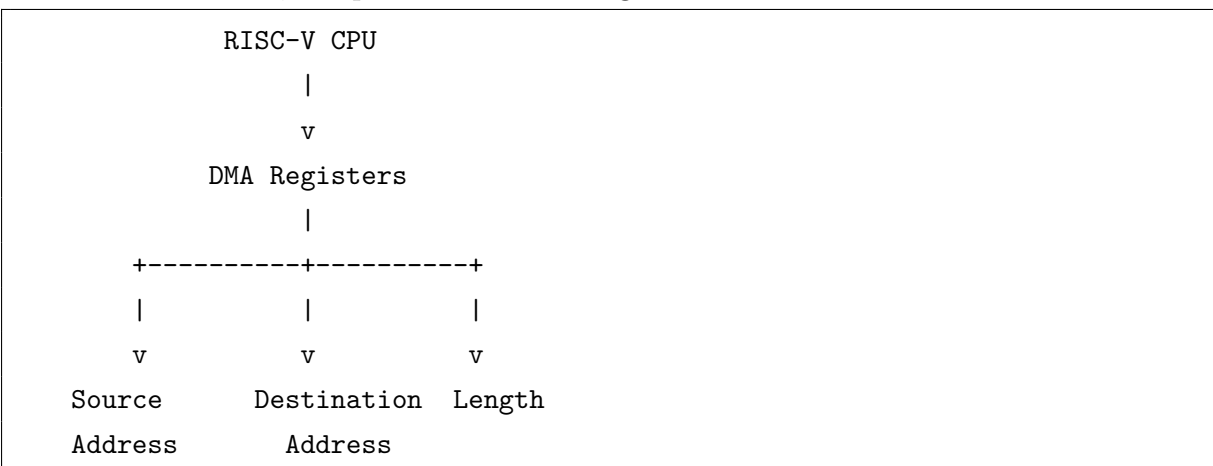
```
RLE_BUSY
RLE_DONE
RLE_ERROR
RLE_RESULT_LEN
```

The basic control sequence is:

```
CPU initializes system
      v
CPU configures RLE
      v
CPU starts RLE
      v
RLE processes data
      v
RLE sets DONE
      v
CPU reads status/result
      v
CPU starts next operation
```

4.2 Control Path for DMA

When DMA is used, the processor first configures the DMA transfer.



The DMA then performs the required block transfer without the CPU having to manually issue a processor operation for every data element.

The processor mainly performs:

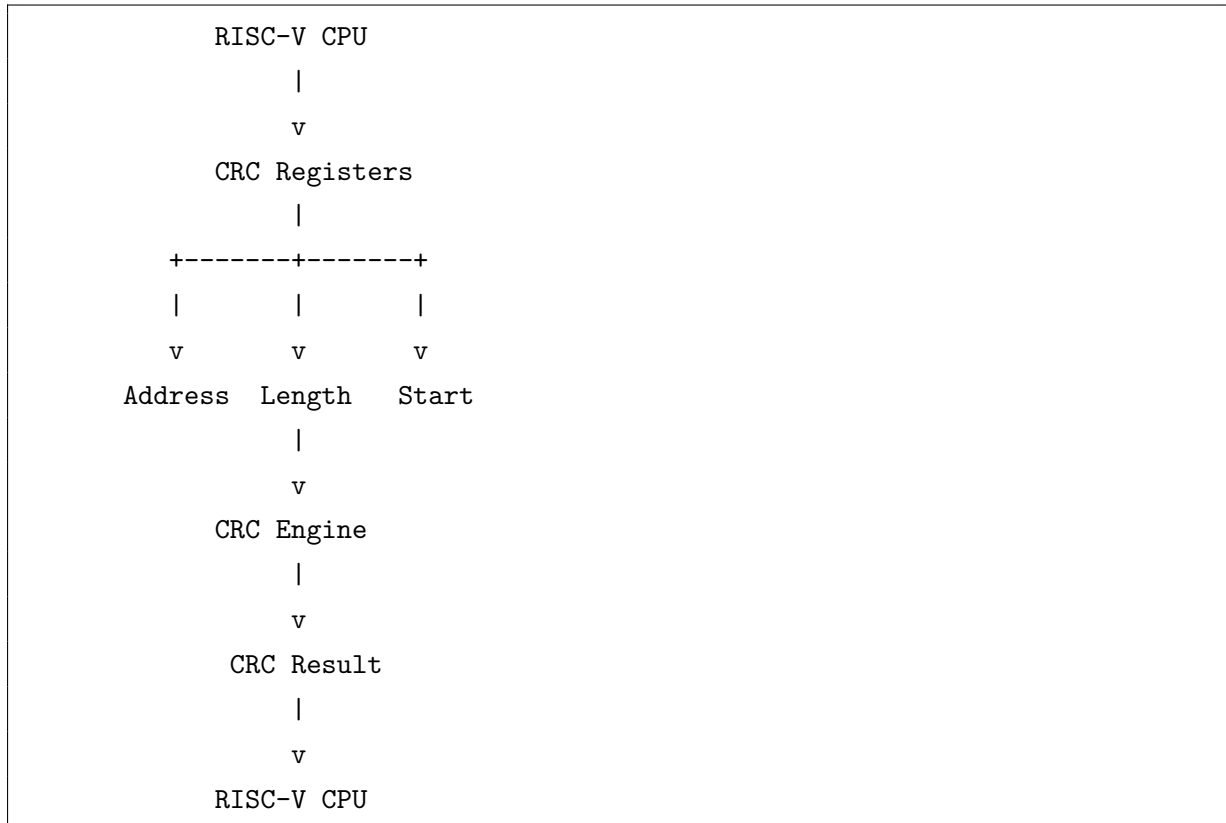
```
Configure -> Start -> Wait/Check -> Continue
```

rather than directly handling every byte.

The exact DMA-to-RLE connection will be finalized when the selected DMA IP and its actual interface are integrated.

4.3 Control Path for CRC

The CRC/checksum engine is also controlled through registers.



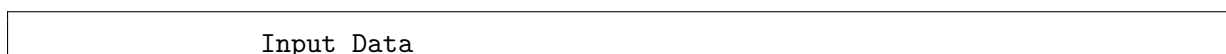
The processor can start a CRC calculation and later read the calculated result.

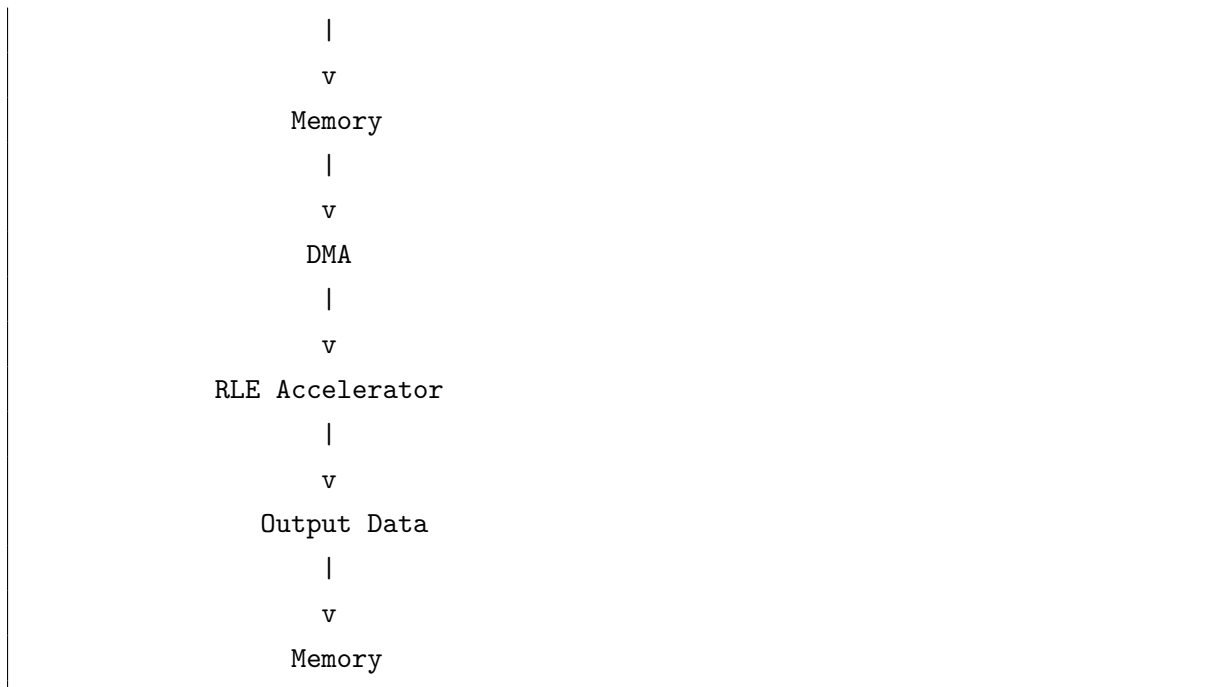
For verification, the software can compare the CRC values of the required data buffers.

5 Data Path

The data path carries the **actual input and output data** through memory, DMA, the RLE accelerator, and the verification logic. The control path configures the operation, while the data path performs the movement and transformation of the actual information.

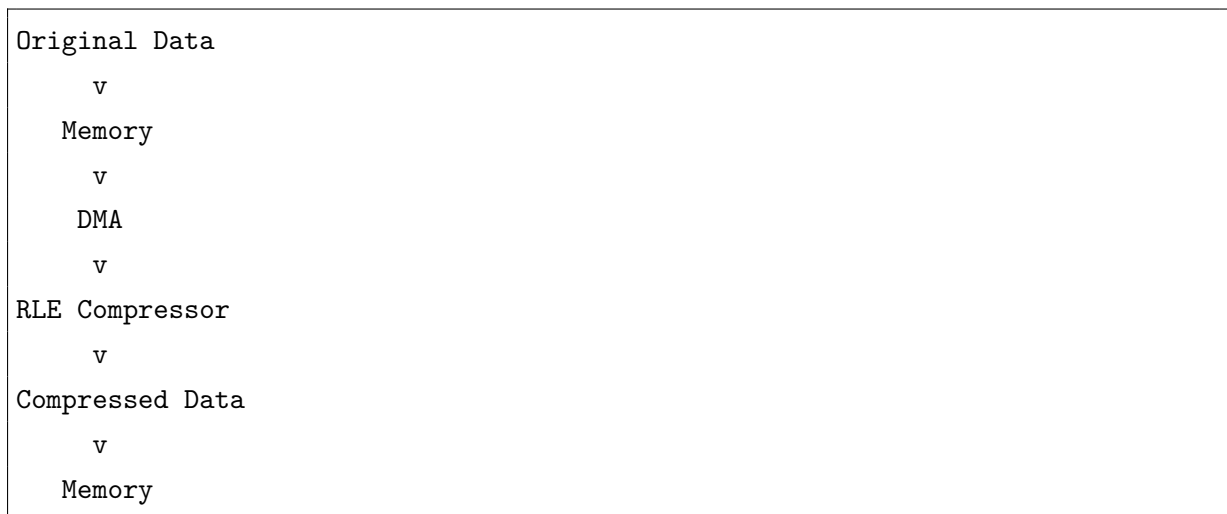
5.1 Main Data Flow





5.2 Compression Data Path

For compression:



5.3 Decompression Data Path

For decompression:





5.4 Data Processing Inside the RLE Accelerator

The RLE accelerator contains the actual compression and decompression logic.

5.4.1 Compression

The input data is processed sequentially.

For example:

A A A A B B C C C

The accelerator internally:

Read A v Compare with next A v Same -> increase counter v Same -> increase counter v Same -> increase counter v B appears v Output A + count v Start counting B

The resulting representation is:

A,4 B,2 C,3

The important internal data-path elements are:

Input Data
v
Input Register
v
Comparator
v
Run Counter
v
Output Logic
v
Output Data

5.4.2 Decompression

For decompression, the accelerator receives a value and its repetition count.

For example:

A,4

The hardware produces:

A A A A

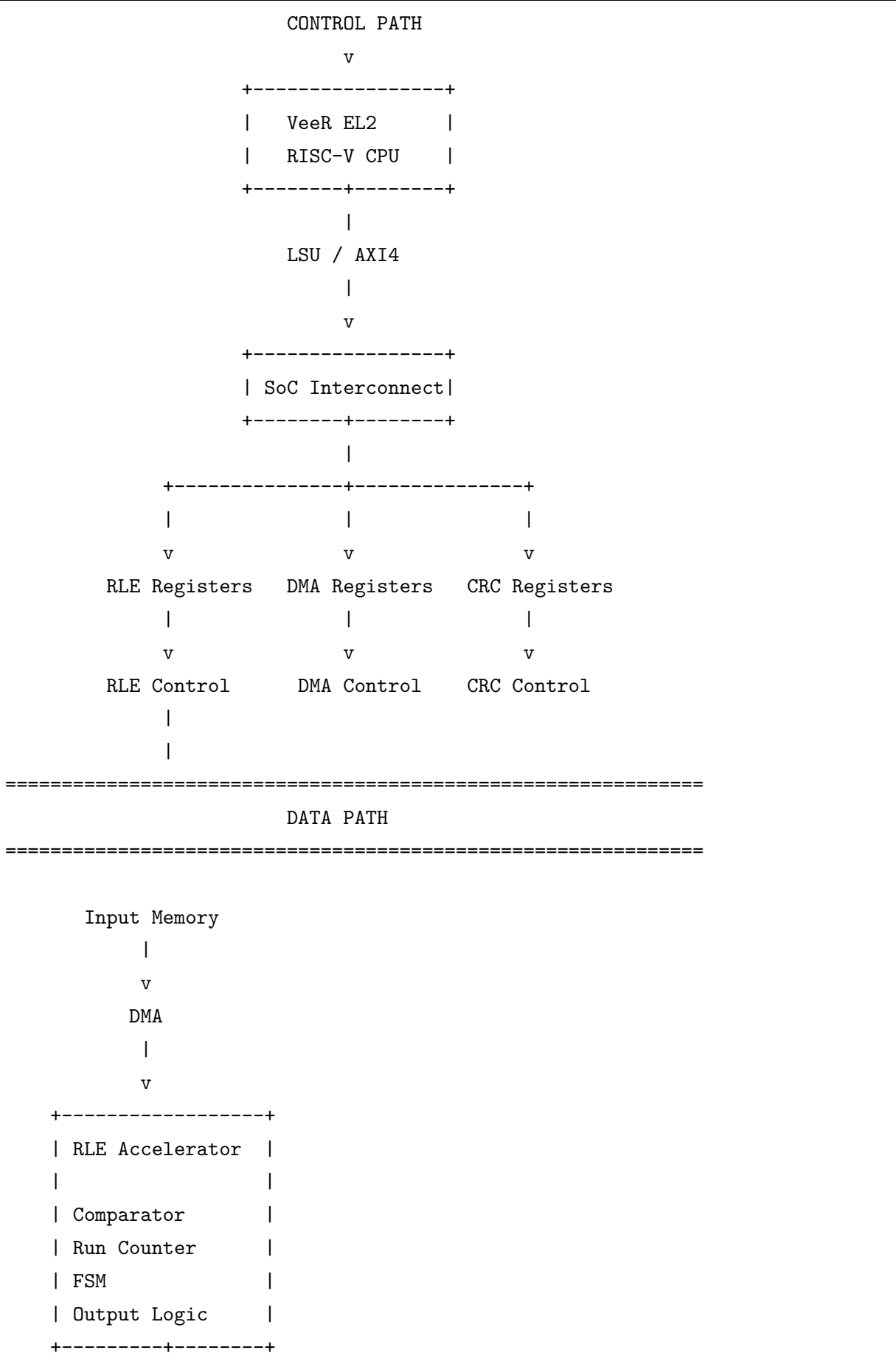
The internal flow is approximately:

Encoded Value
v
Value Register
+
Count Register
v
Output Logic
v
Repeated Output

The counter decreases after every generated output until the count reaches zero.

5.5 Complete Control and Data Path

The overall architecture can therefore be represented as:





5.6 Summary

The control path tells the hardware what operation to perform, while the data path moves the actual data through memory, DMA, and the RLE accelerator.